Course Code: CSA1741 Course Name: ARTIFICIAL INTELLIGENCE Slot: B

ASSIGNMENT 1 – CO1

Nature of Assignment: Individual Duration: 1 Week Total Marks: 50

Student Name: K. Sirisha

Reg. No: 192325130

Task 1 – Design the Agent and the Search Algorithms

1.1 PEAS description

PEAS means **P**erformance measure, **E**nvironment, **A**ctuators and **S**ensors. It tells us clearly what the robot must do and what it has to work with.

Part	**Hospital medicine-delivery robot**
P – Performance	Deliver the correct medicine to the correct ward; keep total travel time as small as possible; no collisions; no wrong or late delivery; save battery.
E – Environment	The hospital building: pharmacy, corridors, junctions, lifts, ward doors. Also patients, nurses and trolleys moving around.
A – Actuators	Drive wheels, steering, lift-button arm, door-opening arm, medicine drawer lock, speaker/beeper.
S – Sensors	Localisation sensor (where am I on the map), obstacle sensors (lidar/ultrasonic), camera, ward-door tag reader (RFID), bumper, battery meter.

1.2 Environment classification

For the route-finding job (Tasks 1–11) we look at the hospital as a map of junctions. Day-4 changes one of these answers (partial observability).

Property	**Answer**	**Why**
Observable	Fully observable (Tasks 1–11)	The localisation sensor tells the robot its exact junction and the map is known. On Day 4 this becomes partially observable.
Deterministic	Deterministic	If the robot chooses “go to junction v”, it ends at v. Travel times are known numbers.

Episodic / Sequential	Sequential	A route is a chain of moves; each move changes where the robot is and so changes every later choice.
Static / Dynamic	Static while planning	The map does not change while the route is being computed. (In real life people walk by, so the robot re-plans if a corridor is blocked.)
Discrete / Continuous	Discrete	We model the building as a graph: a finite set of junctions and corridors, not free movement in a continuous space.
Single / Multi agent	Single agent	Only one robot plans. Other people are treated as obstacles, not as players with their own goals.

1.3 View of AI and why the robot is rational

The robot follows the "acting rationally" view of AI. It does not try to copy human thinking or human behaviour. It simply chooses the action that is expected to give the best result according to its performance measure: the cheapest route to the ward.

It is a rational problem-solving (goal-based) agent because (1) it has a clear goal – the requested ward; (2) it uses a model of the world – the hospital graph; (3) it searches through possible action sequences and picks one that reaches the goal at the lowest cost; (4) it makes the best use of the information it has (perfect position and map).

Link to early AI history. Route finding is the classic "problem solving as search" idea from the 1950s–60s: Newell and Simon's Logic Theorist and General Problem Solver treated problems as states and moves; Moore (1959) described breadth-first search for finding shortest paths and Dijkstra (1959) the cheapest-path method that is the basis of uniform-cost search; and the Shakey robot (1960s) planned routes between rooms by searching, which is very close to our hospital robot.

1.4 Problem formulation

Element	**Definition for the hospital robot**
States	Each location (junction, corridor point, lift stop, ward door). There are V of them: 1,000 → 1,000,000+.
Initial state	The pharmacy junction (node 0).
Actions	Move(u, v) for every corridor that joins the current location u to a neighbour v. At most b actions in each state.
Transition model	RESULT(u, Move(u, v)) = v. The move always works (deterministic).
Goal test	Is the current state equal to the requested ward?

Path cost	Sum of the travel times of the corridors used. Each step cost c(u, v) = w ≥ ε > 0 (in our tests w is 1 to 9 time units, so ε = 1).

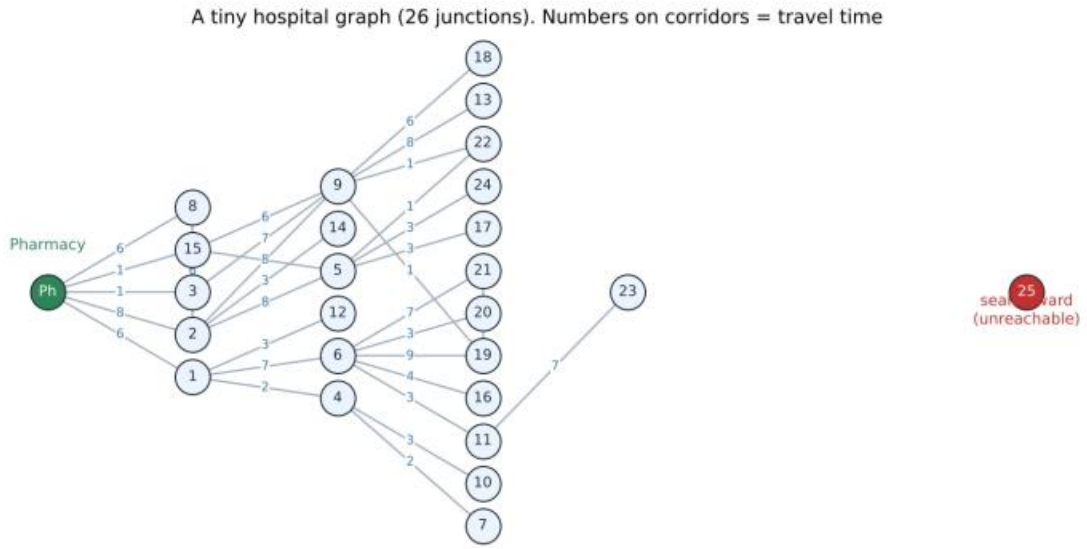


Figure 1 – A tiny example of the hospital graph. The real tests use 1,000 to 1,000,000 junctions built in the same way.

1.5 Three different uninformed strategies

I chose three strategies that are different in how they pick the next node and what they guarantee:

#	Strategy	Style	Basic idea	Why I chose it
1	Breadth-First Search (BFS)	Iterative – queue (FIFO)	Visit all junctions 1 corridor away, then 2 corridors away, and so on.	Simple and fast. Finds the route with the fewest corridors. Good baseline.
2	Uniform-Cost Search (UCS)	Iterative – priority queue	Always expand the junction with the smallest travel time so far.	The robot cares about **travel time**, not number of corridors. UCS is the optimal one.
3	Iterative-Deepening DFS (IDDFS)	**Recursive** – depth-limited DFS repeated with limit 0, 1, 2, …	Go deep along one path up to a limit; if the goal is not found, raise the limit and start again.	Uses very little memory. This is the "memory-saving" option when the hospital becomes huge.

BFS and UCS are both iterative but are not the same idea: BFS ignores travel time and uses a plain queue, while UCS uses a priority queue ordered by cost. IDDFS is a recursive depth-first idea. The iterative and recursive code of one strategy is never counted twice.

Strategy 1 – BFS (pseudocode)

```
BFS(adj, start, goal):
   if start == goal: return [start]
   explored <- { start : none }          # explored set + parent pointers
   frontier <- queue [start]
   while frontier is not empty:
      u <- frontier.pop_front()          # EXPAND u
      for each (v, w) in adj[u]:         # generate successors
         if v not in explored:           # duplicate detection
            explored[v] <- (u, w)
            if v == goal: return path built from parent pointers
            frontier.push_back(v)
   return FAILURE                        # goal unreachable
```

Strategy 2 – UCS (pseudocode)

```
UCS(adj, start, goal):
   frontier <- priority queue [(0, start)] ;  dist[start] <- 0 ;  explored <- {}
   while frontier is not empty:
      (g, u) <- frontier.pop_min()       # cheapest node first
      if u in explored: continue         # skip old, more expensive copy
      if u == goal: return path          # goal test when EXPANDED (keeps it optimal)
      explored.add(u)
      for each (v, w) in adj[u]:
         if v not in dist or g + w < dist[v]:
            dist[v] <- g + w ;  parent[v] <- u
            frontier.push((g + w, v))
   return FAILURE
```

Strategy 3 – IDDFS with recursive DLS (pseudocode)

```
IDDFS(adj, start, goal, L_max):
   for limit = 1, 2, 3, ..., L_max:
      result <- DLS(start, goal, limit)
      if result is a path: return result
   return FAILURE / "gave up at L_max"
```

```
DLS(u, goal, rem):                    # recursive, rem = remaining depth
  if rem == 0: return CUTOFF
  for each (v, w) in adj[u]:
    if v is on the current path: continue      # avoid going round in circles
    if v == goal: return [u, v]
    r <- DLS(v, goal, rem - 1)                 # recursive call
    if r is a path: return [u] + r
  return CUTOFF or FAILURE
```

1.6 Preprocessing needed

- **Adjacency list (all three):** the corridors arrive as a list of pairs (u, v, w). We turn them into adj[u] = [(v, w), …] so that "all neighbours of u" can be found at once. Cost: one pass over all corridors, **Θ(V + E)**, done once and reused for every delivery.
- **Explored set + parent pointers (BFS, UCS):** a hash table that stores every junction already reached. It stops the robot visiting the same junction twice and lets us rebuild the route. Memory: up to V entries.
- **Distance table (UCS):** best cost found so far for each junction, to decide whether a new route is cheaper.
- **Current-path set (IDDFS):** only the junctions on the current path, so loops are avoided. Memory is just the depth.
- **Optional "deepest-remaining" table (IDDFS-memo):** a modification I also test. It remembers how much depth was left when each junction was expanded so the same junction is not expanded again with less depth. It repairs the repeated work of IDDFS but costs V memory.

Evidence for Task 1: PEAS and classification (1.1–1.2), view of AI and history (1.3), formulation (1.4), three strategies with pseudocode, reasons and preprocessing (1.5–1.6).

Task 2 – Identify the Basic Operation

The basic operation is the step that is repeated the most and so decides the running time. If we count how many times it runs, we know how the time grows.

Algorithm	Basic operation	Where it happens	Why it is a good choice	How the count grows with state space

BFS	**Successor generation with a duplicate check**: looking at a corridor (u → v), testing v not in explored, and adding v to the queue.	Inner for loop inside the expansion of u.	Everything else (queue pop, parent save) happens at most once per generated node, so the loop body runs at least as often as them all.	Each junction is expanded once and each corridor is looked at twice (once from each end): about 2E checks. Grows linearly with V. Without duplicate detection it would grow like b^d.
UCS	Frontier insertion/removal in the priority queue (heap push and pop) together with the cost comparison g + w < dist[v].	Pop at the top of the while loop; push inside the for loop.	Each push/pop costs O(log n) – more than any other single step – so these decide the time.	Pushes ≈ number of corridors scanned: a little more than V (we measured 1.12 V). Every node with cost below C* is expanded, so for a long cheap path the count is like $b^{1+\lfloor C^*/\varepsilon \rfloor}$.
IDDFS	Recursive call of DLS (node generation/visit), which includes goal test and loop over successors.	The call DLS(v, goal, rem-1) inside the loop.	Every piece of work (goal test, path check, loop) happens once per call, so the number of calls measures the time.	The top levels are visited again in every iteration. Calls = Σ over limits of ($b + b^2 + \ldots + b^l$). Grows exponentially with depth, so it explodes as hospital depth grows.

In simple words. For BFS the work is "look at every corridor once", so doubling the hospital doubles the work. For UCS it is the same plus the extra cost of keeping the heap sorted. For IDDFS the work is "walk every path up to the depth limit, again and again", so a small increase in depth multiplies the work.

Evidence for Task 2: basic operation, location, justification and growth given for each of the three algorithms.

Task 3 – Derive the Time and Space Complexity

3.1 Input size and assumptions

- b = branching factor (corridors that leave a junction, on average); d = depth (number of corridors) of the shallowest goal; m = maximum depth; C* = cost of the cheapest route; ε = smallest corridor cost; V and E = numbers of junctions and corridors.
- For the derivation we use the standard search tree: level i has b^i nodes. Then we correct for the real graph (finite V, explored set).
- Goal test is done when a node is generated for BFS and IDDFS, and when it is expanded for UCS (as in the code).

3.2 BFS

Number of basic operations. Level 1 has b nodes, level 2 has b^2, …, level d has b^d. In the worst case the goal is the last node of level d, so every node up to level d is generated:

$T_{BFS}(d) = b + b^2 + b^3 + \ldots + b^d = b(b^d - 1)/(b - 1) = O(b^d)$

(If the goal test were done at expansion, one more level is generated and the bound would be $O(b^{d+1})$.) For example, b = 3, d = 4 gives 3 + 9 + 27 + 81 = **120** nodes.

Space. All nodes of the last level wait in the frontier (b^d) and the explored set holds all earlier levels (about b^{d-1}). So

$S_{BFS}(d) = O(b^d)$

Real graph. There are only V distinct junctions and the explored set stops repeats, so each junction is expanded at most once and each corridor is looked at twice: $T = \Theta(V + E)$ at worst, and $S = \Theta(V)$. Therefore the operation count is $O(\min(b^d, V + E))$.

Preprocessing. Building the adjacency list costs $\Theta(V + E)$. **Overall:** $T_{total} = \Theta(V + E) + O(\min(b^d, V + E)) =$ **$\Theta(V + E)$**. Preprocessing is of the same order as the worst-case search, so it must not be ignored (we measured it: 1.99 s versus 3.17 s search at 1,000,000 states).

3.3 UCS

UCS expands nodes in order of path cost, so it expands every node whose cost is less than C* (the cost of the best route). Every corridor costs at least ε, so a path of cost C* has at most $\lfloor C^*/\varepsilon \rfloor$ corridors, and the goal sits at depth about $1 + \lfloor C^*/\varepsilon \rfloor$ in the cost-ordered tree.

$T_{UCS} = \Sigma_{i=0}^{1+\lfloor C^*/\varepsilon \rfloor} b^i = O(b^{1+\lfloor C^*/\varepsilon \rfloor})$ node expansions

Each push and pop in the heap costs O(log n), where n is the frontier size, so the time is $O(b^{1+\lfloor C^*/\varepsilon \rfloor} \cdot \log n)$. The frontier and the explored set store the same nodes:

$S_{UCS} = O(b^{1+\lfloor C^*/\varepsilon \rfloor})$

Real graph. The explored set limits expansions to V, and each corridor causes at most one push, so $T = O((V + E) \log V)$ and $S = \Theta(V)$. **Preprocessing:** $\Theta(V + E)$. **Overall:** $\Theta(V + E) + O((V + E) \log V) =$ **$O((V + E) \log V)$**. When all costs are equal, ε is the step cost and $C^*/\varepsilon = d$, so UCS behaves like BFS, with the extra log factor.

3.4 IDDFS (recursive)

Iteration with limit l is a depth-limited search that generates $b + b^2 + \ldots + b^l$ nodes. The root is generated once. The search stops at the iteration whose limit equals d.

$T_{IDDFS}(d) = \Sigma_{l=1}^{d} \Sigma_{i=1}^{l} b^i$

Swap the sums: a node at level i is generated in every iteration with limit $l \geq i$, that is $(d - i + 1)$ times. So

$T_{IDDFS}(d) = \Sigma_{i=1}^{d} (d - i + 1)\, b^i = d \cdot b + (d-1) \cdot b^2 + \ldots + 2 \cdot b^{d-1} + 1 \cdot b^d$

Take b^d out and use $\Sigma\, (j+1)\, x^j = 1/(1 - x)^2$ for $x = 1/b$:

$T_{IDDFS}(d) = b^d \Sigma_{j=0}^{d-1} (j + 1)\, b^{-j} \leq b^d \cdot 1/(1 - 1/b)^2 = O(b^d)$

For b = 3, d = 4: 4·3 + 3·9 + 2·27 + 1·81 = **174** nodes, against 120 for BFS: only about 45% extra (the limit is b/(b − 1) = 1.5 for a single last iteration compared with BFS). The repeated work is small because the **last level is the biggest**.

Space. Only one path (at most d nodes) plus the current-path set is kept at a time:

$S_{IDDFS}(d) = O(d)$ (O(b·d) if all siblings of the path are stored)

Preprocessing: adjacency list Θ(V + E). **Overall:** Θ(V + E) + $O(b^d)$. Here the search can cost far more than the preprocessing. With the "memo" table, each iteration visits each junction about once, so T = O(d (V + E)) and S = Θ(V).

3.5 Summary of the derived functions

Algorithm	Basic-operation count (time)	Memory (frontier / stack)	Preprocessing	Overall
BFS	$b(b^d-1)/(b-1) \rightarrow \Theta(V+E)$ on a real graph	$O(b^d) \rightarrow \Theta(V)$	Θ(V+E)	Θ(V+E)
UCS	$O(b^{1+\lfloor C^*/\varepsilon \rfloor}) \rightarrow O((V+E) \log V)$	$O(b^{1+\lfloor C^*/\varepsilon \rfloor}) \rightarrow \Theta(V)$	Θ(V+E)	O((V+E) log V)
IDDFS	$\Sigma\ (d-i+1)\ b^i \le b^d/(1-1/b)^2$	O(d)	Θ(V+E)	Θ(V+E) + $O(b^d)$
IDDFS-memo	O(d (V+E))	Θ(V)	Θ(V+E)	O(d (V+E))

Evidence for Task 3: inputs b, d, m, C*, ε defined; operation counts derived level by level; time, space, preprocessing and overall cost for every algorithm.

Task 4 – Best, Average and Worst Case

The same algorithm can behave very differently depending on where the goal is. Below, "nodes generated" is the quantity we count (the numbers come from the real run in Task 9, with V = 10,000 and 100,000 junctions).

4.1 BFS

- **Best case:** the goal is a direct neighbour of the pharmacy and is generated first. Only 1 expansion and at most b nodes: **Θ(1)**.
- **Average case:** the goal is at a typical depth d. All levels before it are generated, and on average half of level d: about $(b^d - b)/(b - 1) + b^d/2$ nodes – for b = 3 about b^d. In a real graph most junctions are in the last levels, so BFS has to generate roughly half of all junctions (measured: 4,403 of 10,000 = 44%).
- **Worst case:** the goal is the last node of the deepest level – or unreachable. Then every junction is generated once: all V nodes (measured 9,999 and 99,999).

4.2 UCS

- **Best case:** the goal is reached through the cheapest corridor from the pharmacy, so it is the first one popped: only a handful of nodes (measured 12 and 13).

- **Average case:** the goal has cost g. Every junction with cost below g is expanded first, so the work depends on how many junctions are "cheaper than the goal" – measured 7,523 (75% of V) and 79,639 (80%).
- **Worst case:** (a) **cheap-edge detour** – the best route is made of many cheap corridors while a direct corridor is expensive; UCS must first expand everything cheaper, up to $b^{1+\lfloor C^*/\varepsilon \rfloor}$ nodes; (b) goal unreachable – all V junctions are expanded and about 12% extra pushes happen (11,187 for V = 10,000).

4.3 IDDFS (recursive)

- **Best case:** the goal is a neighbour of the pharmacy. It is found in iteration 1: **Θ(1)**.
- **Average case:** the goal is at depth d. The iterations before d are done fully, and the last iteration is stopped half-way on average: about $T(d-1) + \frac{1}{2} b^d$ nodes. Measured 11,222 and 287,986.
- **Worst case:** the goal is the last branch tried at depth d: the full $\Sigma (d - i + 1) b^i$. If the goal is unreachable the pure IDDFS never finishes by itself – it must be stopped by a depth cap L_{max}, costing $O(b^{L_\{max\}})$. We saw this clearly: for the sealed ward the pure version hit our 5,000,000-node limit even with only 1,000 junctions. The memo version stops by itself (it stops when a new, bigger limit finds no new junction) after 180,039 nodes at V = 10,000.

4.4 Measured evidence

```
Nodes generated (start = pharmacy). best = goal 1 corridor away; avg = mean of 30 random wards;
worst = farthest ward, and unreachable sealed ward
       n algorithm    best    average   worst: far  worst: unreachable
  10,000 BFS             7      4,403        9,999              9,999
  10,000 UCS            12      7,523       11,186             11,187
  10,000 IDDFS-tree      7     11,222    1,701,373   > 5,000,000 (cap)
  10,000 IDDFS-memo      7      8,707      129,079            180,039
 100,000 BFS             3     48,601       99,982             99,999
 100,000 UCS            13     79,639      111,895            111,984
 100,000 IDDFS-tree      3    287,986  > 5,000,000   > 5,000,000 (cap)
 100,000 IDDFS-memo      3    136,976    1,320,893          1,975,284
```

Visual Studio Code terminal – real output – report_cases.py

What the table shows. BFS and UCS stay within about 1–1.1 V nodes even in the worst case, because the explored set prevents repeats. The pure IDDFS has a very small best case but a terrible worst case: 1,701,373 nodes for 10,000 junctions (170 times V) and more than 5,000,000 at 100,000 junctions – the nodes are generated again and again via different routes.

Evidence for Task 4: conditions that create the best, average and worst cases for each algorithm, how the nodes generated change and why, with measured numbers.

Task 5 – Asymptotic Analysis [3 Marks]

Notation: **O** is an upper limit, **Ω** a lower limit, **Θ** means both (exact growth rate).

Algorithm	Time	Memory	Meaning for the hospital robot

BFS	O(V+E); worst case Θ(V+E); best Ω(1)	Θ(V) worst case	Double the hospital → double the time and double the memory. Predictable.
UCS	O((V+E) log V); best Ω(1)	Θ(V)	Same growth as BFS, with a small log factor (about 2–3× slower in our tests).
IDDFS (pure)	$O(b^d)$, exponential; Ω(1)	Θ(d) – very small	Memory never limits it, but time explodes quickly when the hospital gets deeper or more branchy.
IDDFS-memo	O(d (V+E))	Θ(V)	Time grows like V times the depth. Cannot beat BFS/UCS but is no longer exponential.

5.1 How they behave when the hospital grows from 1,000 to 1,000,000+ states

Using the farthest-ward measurements (full numbers in Task 9):

States V	**BFS nodes**	**BFS max frontier**	**UCS nodes**	**IDDFS-pure nodes**	**IDDFS-pure memory**
1,000	998	344	1,115	35,172	8
10,000	9,999	3,268	11,186	1,701,373	11
100,000	99,982	31,432	111,895	> 5,000,000	13 (= depth)
1,000,000	999,991	313,618	1,121,087	> 5,000,000	15 (= depth)

- **BFS and UCS (Θ(V)):** when V grows ×10, time and nodes grow ×10, and the frontier is about 31–34% of V. Both time and memory are linear, but memory is the one that a small on-board computer runs out of first: at 1,000,000 states BFS must hold 313,618 junctions in the frontier plus nearly 1,000,000 in the explored set.
- **IDDFS-pure (exponential time, tiny memory):** memory is only the path length (8 → 15), but time grows ×48 when V grows ×10 and stops being usable already at 100,000 states. Time is the limiting factor first for this algorithm.
- **Conclusion:** BFS/UCS are limited by memory (and by time at the very end); IDDFS is limited by time. As the hospital grows from 1,000 to 1,000,000+ states, the Θ(V) algorithms stay practical, but the exponential one does not.

Evidence for Task 5: correct O/Ω/Θ for time and memory, comparison of growth, and what it means when the hospital grows to 1,000,000+ states.

Task 6 – Formulate the Recurrence Relation

The recursive algorithm is the depth-limited search DLS(u, goal, rem) inside IDDFS. The recurrence is written by looking at what this function does:

```
DLS(u, goal, rem):
  if rem == 0: return CUTOFF              # base case  - no more recursion
  for each (v, w) in adj[u]:              # b successors
    if v on current path: continue        # constant work
    if v == goal: return [u, v]           # constant work
    r <- DLS(v, goal, rem - 1)            # one recursive call per successor
    ...
```

Item asked in the question	**Answer taken from the algorithm**
Size of the original problem	The remaining depth l = rem (the depth limit of this call).
Size of the sub-problem	l − 1: the call for the neighbour has one less step to go.
Number of recursive calls	b, one for each corridor that leaves the junction (the branching factor). See the recursion tree in Figure 2.
Work outside the recursive calls	A constant c: goal test, current-path check, add/remove from the path set and loop overhead.
Base case	T(0) = 1: when rem = 0 the function returns CUTOFF immediately (one call, no further work).

Putting these together: T(l) is the number of calls made by DLS with remaining depth l. One call does c work, then makes b calls with depth l − 1:

$T(l) = b \cdot T(l - 1) + c, \quad T(0) = 1$

For the memory used by the stack, each call adds one frame and then calls with l − 1, giving the second recurrence $S(l) = S(l - 1) + 1$ with $S(0) = 1$.

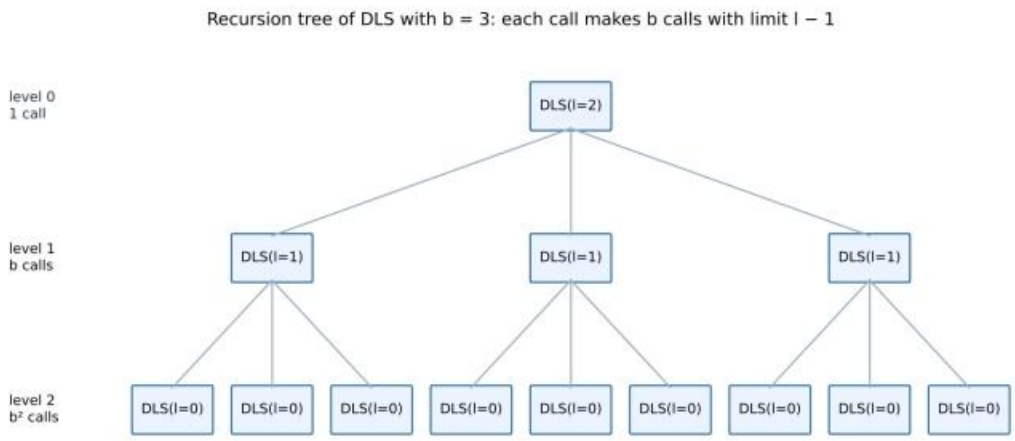


Figure 2 – Recursion tree for DLS with b = 3. Level i has b^i calls, which is where the recurrence comes from.

Quick check of the recurrence on a real run

I built a perfect 3-ary tree (every junction has 3 children) with no goal, ran the real code, and counted the calls made in each iteration. With c counted as 1 per call the recurrence gives $b + b^2 + \ldots + b^l$, which matches exactly:

```
# Check T(l) = b*T(l-1) + c on a perfect tree: count DLS calls and compare with the closed form.
from hospital_search import iddfs
b = 3
def tree(depth):                                    # perfect b-ary tree, node ids in heap order
    n = sum(b**i for i in range(depth + 1)); adj = [[] for _ in range(n)]
    for v in range(1, n): u = (v - 1) // b; adj[u].append((v, 1)); adj[v].append((u, 1))
    return adj, n
adj, n = tree(7)
print(f"perfect {b}-ary tree, {n} nodes, goal does not exist")
print(f"{'limit l':>8}{'calls measured (this iteration)':>34}{'formula b+...+b^l':>20}")
prev = 1
for l in range(1, 7):
    r = iddfs(adj, 0, n + 5, max_limit=l)           # goal absent -> runs limits 1..l
    total = r[2].generated - prev; prev = r[2].generated
    print(f"{l:>8}{total:>34,}{sum(b**i for i in range(1, l+1)):>20,}")
```

Visual Studio Code – real code – check_recurrence.py

```
perfect 3-ary tree, 3280 nodes, goal does not exist
 limit l   calls measured (this iteration)   formula b+...+b^l
       1                                 3                  3
       2                                12                 12
       3                                39                 39
       4                               120                120
       5                               363                363
       6                             1,092              1,092
```

Visual Studio Code terminal – real output – output of check_recurrence.py

Evidence for Task 6: recurrence T(l) = b·T(l−1) + c with T(0) = 1; all five components identified from the code; verified by running it.

Task 7 – Solve the Recurrence Relation

7.1 Why this method

The recurrence reduces the size only by 1 (subtract-and-conquer), not by dividing it. So the Master Theorem (which needs n/b) does not apply. The best choice is backward substitution (iteration), checked by induction (substitution), with the recursion tree (Figure 2) giving the picture.

7.2 Steps

Start from $T(l) = b\,T(l-1) + c$ and keep substituting:

$T(l) = b\,[\,b\,T(l-2) + c\,] + c = b^2\,T(l-2) + c\,(b+1)$

$T(l) = b^3\,T(l-3) + c\,(b^2 + b + 1)$

… after k substitutions: $T(l) = b^k\,T(l-k) + c\,(b^{k-1} + \ldots + b + 1)$

Stop when $k = l$ (reach the base case $T(0) = 1$):

$T(l) = b^l \cdot 1 + c\,(b^{l-1} + \ldots + b + 1) = b^l + c\,(b^l - 1)/(b-1)$

$T(l) = (1 + c/(b-1)) \cdot b^l - c/(b-1)$

Check by induction. Base: $l = 0$ gives $(1 + c/(b-1)) - c/(b-1) = 1 = T(0)$ ✓. Step: assume the formula for $l - 1$. Then $b \cdot T(l-1) + c = (1 + c/(b-1))\,b^l - bc/(b-1) + c = (1 + c/(b-1))\,b^l - c/(b-1)$ ✓ (because $-bc/(b-1) + c = -c/(b-1)$).

7.3 Result and meaning

$T(l) = \Theta(b^l)$ and, with $l = d$, $T(d) = \Theta(b^d)$ $S(l) = \Theta(l)$

- **Depth:** each extra corridor of depth multiplies the time by b. With $b = 3$, one more level triples the work.
- **Branching:** a bigger b makes the base of the exponent bigger, so hospital corridors with many branches (hubs, big junctions) hurt much more than a longer straight corridor.
- **Memory:** only $\Theta(l)$ – linear in the depth. This is the strong point of the recursive method.

7.4 Extension to iterative deepening

IDDFS calls DLS for $l = 0, 1, \ldots, d$, so its total time is the sum of the solved recurrence:

$T_{ID}(d) = \Sigma_{l=0}^{d}\,T(l) = (1 + c/(b-1)) \cdot (b^{d+1} - 1)/(b-1) - (d+1)\,c/(b-1) = \Theta(b^d)$

Compared with a single DLS(d), $T_{ID}(d)/T(d) \to b/(b-1)$ (1.5 for $b = 3$; 2 for $b = 2$). So repeating the search costs only a constant factor – the cost of the earlier iterations is small compared to the last one – while the memory stays $\Theta(d)$. IDDFS therefore has the same time order as BFS, $O(b^d)$, but with far less memory.

Evidence for Task 7: method justified; substitution steps shown; result $\Theta(b^l)$; proof by induction; extension to IDDFS $\Sigma\,T(l) = \Theta(b^d)$.

Task 8 – Implement and Test the Algorithms

8.1 Test set-up

- **Graphs:** generated by make_hospital_edges(n, seed) with n = 1,000; 10,000; 100,000; 1,000,000 junctions. Junction 0 is the pharmacy. Each new junction is joined to a random earlier junction (so the graph is connected), then $0.5 \cdot n$ extra random corridors/lifts are added

to make loops. Corridor travel times are random integers 1–9, so ε = 1. The last junction (n − 1) has no corridor: a sealed ward used as the unreachable goal.

- **Seeds (reported):** graph seeds 11, 22, 33, 44 for the four sizes; goal-picking seed 2024 + n. All algorithms use the same graph and the same start/goal pair.
- **Test cases for every size:** (1) reachable goal, (2) unreachable goal (sealed ward), (3) start = goal. Plus the stress case “farthest ward”.
- **Correctness check:** every returned route is re-checked by valid() – it must start at the start, end at the goal, use real corridors, and its cost must equal the sum of the corridor costs. The program stops with an error if any path is wrong (it did not).
- **Python 3.13**, standard library only (heapq, collections.deque, random, time).
- **Reproducible procedure:** python3 run_experiments.py → writes results.json; then python3 report_tables.py, report_tests.py, report_cases.py print the tables below. python3 belief_search.py runs Day-4.

8.2 Code: the three algorithms (hospital_search.py)

```
"""Hospital medicine-delivery robot: three uninformed search strategies.
   1. BFS   (iterative, queue)            2. UCS (iterative, priority queue)
   3. IDDFS (recursive depth-limited search, repeated with growing limit)"""
import random, heapq, sys
from collections import deque

sys.setrecursionlimit(10000)

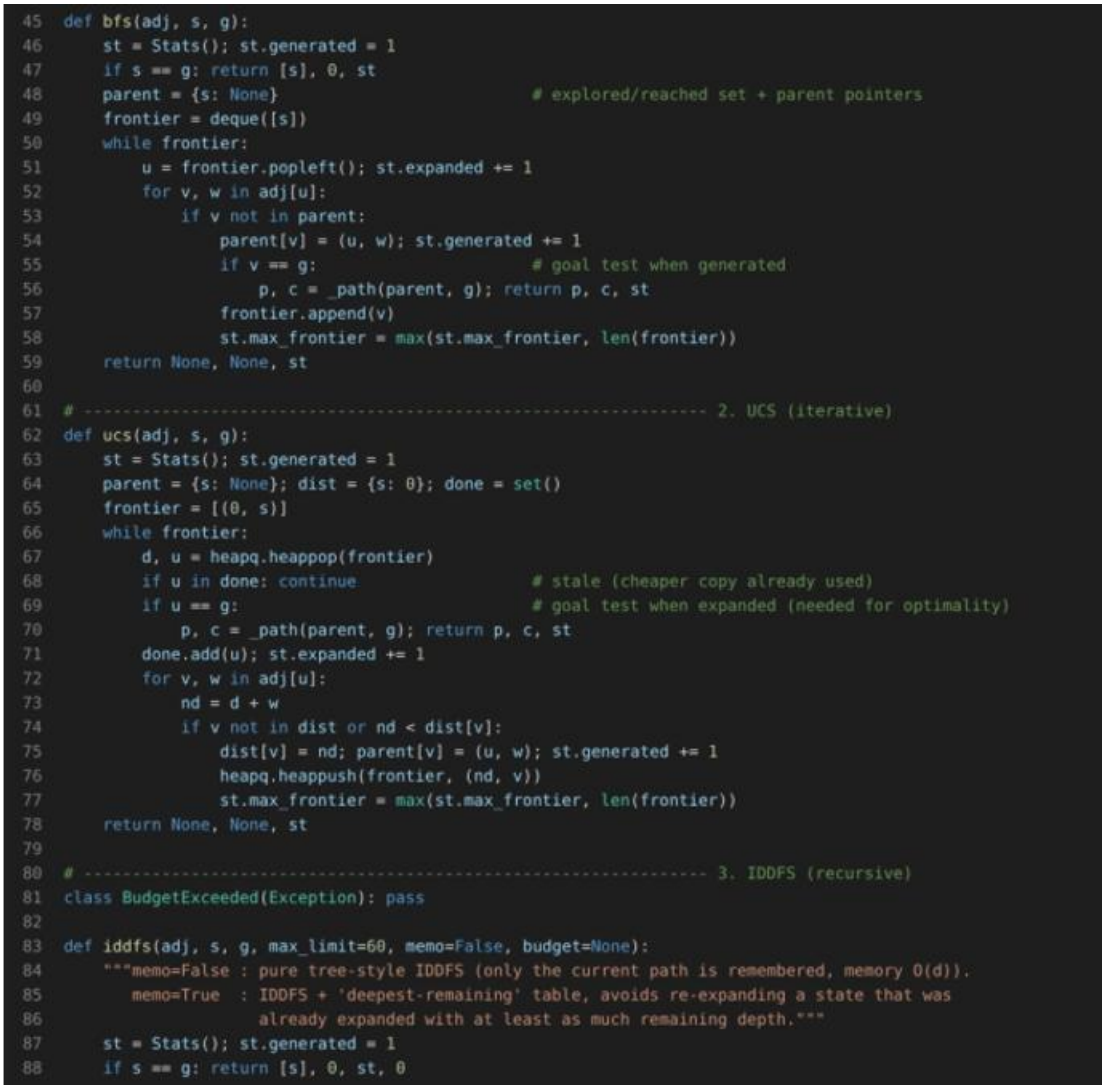
# ------------------------------------------------------------- graph generator
def make_hospital_edges(n, seed, extra=0.5, max_w=9):
    """Junction 0 = pharmacy. Junctions 0..n-2 form a connected hospital;
    junction n-1 is a sealed ward (no corridor) -> the 'unreachable' test case."""
    rng = random.Random(seed)
    m = n - 1
    edges, seen = [], set()
    for v in range(1, m):                    # corridor tree: every junction hangs off an earlier one
        u = rng.randrange(v)
        edges.append((u, v, rng.randint(1, max_w))); seen.add((u, v))
    for _ in range(int(extra * m)):          # extra corridors / lifts create loops
        u, v = rng.randrange(m), rng.randrange(m)
        if u != v and (min(u, v), max(u, v)) not in seen:
            seen.add((min(u, v), max(u, v)))
            edges.append((u, v, rng.randint(1, max_w)))
    return edges

def build_adjacency(n, edges):
    """PREPROCESSING: edge list -> adjacency list.  Cost Theta(V + E)."""
    adj = [[] for _ in range(n)]
    for u, v, w in edges:
        adj[u].append((v, w)); adj[v].append((u, w))
    return adj

class Stats:
    def __init__(self): self.generated = 0; self.expanded = 0; self.max_frontier = 0

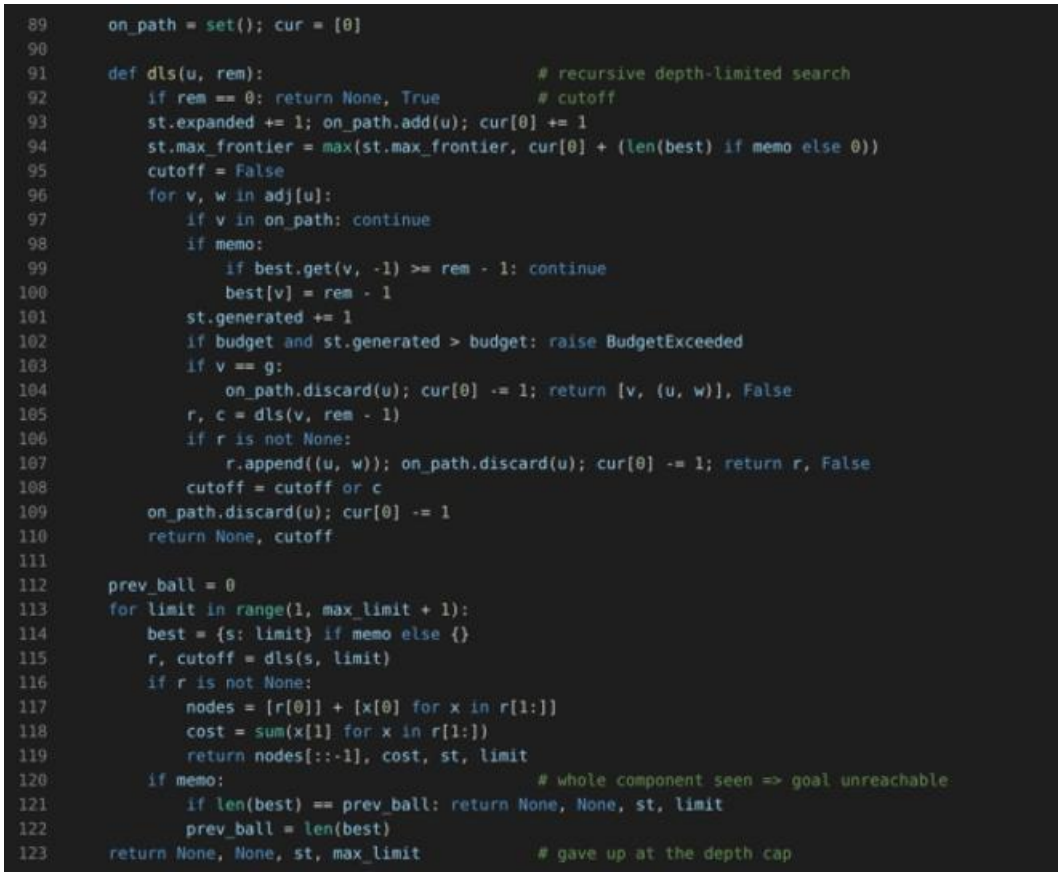
def _path(parent, g):
    path, cost, v = [], 0, g
    while v is not None:
        path.append(v); p = parent[v]
        if p is not None: cost += p[1]
        v = None if p is None else p[0]
    return path[::-1], cost
```

Visual Studio Code – real code – hospital_search.py (Lines 1–44)



```
45  def bfs(adj, s, g):
46      st = Stats(); st.generated = 1
47      if s == g: return [s], 0, st
48      parent = {s: None}                          # explored/reached set + parent pointers
49      frontier = deque([s])
50      while frontier:
51          u = frontier.popleft(); st.expanded += 1
52          for v, w in adj[u]:
53              if v not in parent:
54                  parent[v] = (u, w); st.generated += 1
55                  if v == g:                      # goal test when generated
56                      p, c = _path(parent, g); return p, c, st
57                  frontier.append(v)
58                  st.max_frontier = max(st.max_frontier, len(frontier))
59      return None, None, st
60
61  # ------------------------------------------------------------ 2. UCS (iterative)
62  def ucs(adj, s, g):
63      st = Stats(); st.generated = 1
64      parent = {s: None}; dist = {s: 0}; done = set()
65      frontier = [(0, s)]
66      while frontier:
67          d, u = heapq.heappop(frontier)
68          if u in done: continue                  # stale (cheaper copy already used)
69          if u == g:                              # goal test when expanded (needed for optimality)
70              p, c = _path(parent, g); return p, c, st
71          done.add(u); st.expanded += 1
72          for v, w in adj[u]:
73              nd = d + w
74              if v not in dist or nd < dist[v]:
75                  dist[v] = nd; parent[v] = (u, w); st.generated += 1
76                  heapq.heappush(frontier, (nd, v))
77                  st.max_frontier = max(st.max_frontier, len(frontier))
78      return None, None, st
79
80  # ------------------------------------------------------------ 3. IDDFS (recursive)
81  class BudgetExceeded(Exception): pass
82
83  def iddfs(adj, s, g, max_limit=60, memo=False, budget=None):
84      """memo=False : pure tree-style IDDFS (only the current path is remembered, memory O(d)).
85         memo=True  : IDDFS + 'deepest-remaining' table, avoids re-expanding a state that was
86                      already expanded with at least as much remaining depth."""
87      st = Stats(); st.generated = 1
88      if s == g: return [s], 0, st, 0
```

Visual Studio Code – real code – hospital_search.py (Lines 45–88)



```
    on_path = set(); cur = [0]

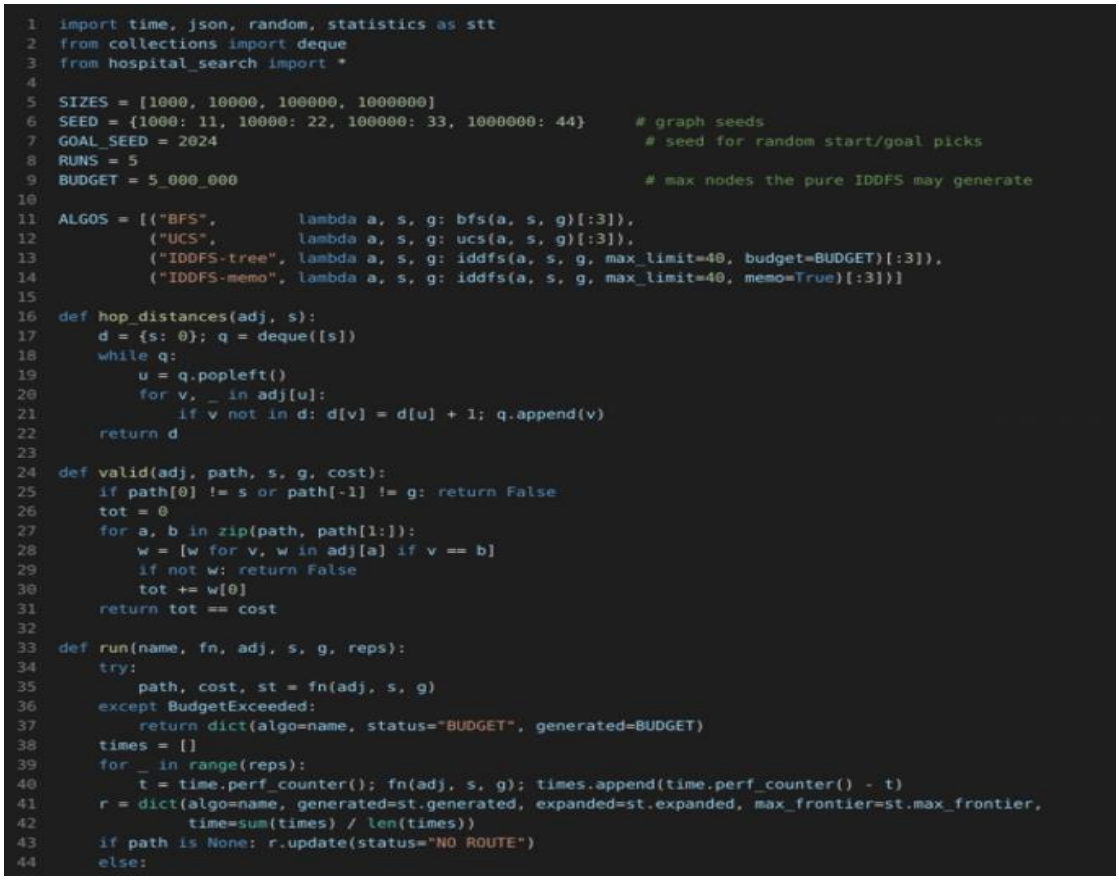
    def dls(u, rem):                          # recursive depth-limited search
        if rem == 0: return None, True        # cutoff
        st.expanded += 1; on_path.add(u); cur[0] += 1
        st.max_frontier = max(st.max_frontier, cur[0] + (len(best) if memo else 0))
        cutoff = False
        for v, w in adj[u]:
            if v in on_path: continue
            if memo:
                if best.get(v, -1) >= rem - 1: continue
                best[v] = rem - 1
            st.generated += 1
            if budget and st.generated > budget: raise BudgetExceeded
            if v == g:
                on_path.discard(u); cur[0] -= 1; return [v, (u, w)], False
            r, c = dls(v, rem - 1)
            if r is not None:
                r.append((u, w)); on_path.discard(u); cur[0] -= 1; return r, False
            cutoff = cutoff or c
        on_path.discard(u); cur[0] -= 1
        return None, cutoff

    prev_ball = 0
    for limit in range(1, max_limit + 1):
        best = {s: limit} if memo else {}
        r, cutoff = dls(s, limit)
        if r is not None:
            nodes = [r[0]] + [x[0] for x in r[1:]]
            cost = sum(x[1] for x in r[1:])
            return nodes[::-1], cost, st, limit
        if memo:                              # whole component seen => goal unreachable
            if len(best) == prev_ball: return None, None, st, limit
            prev_ball = len(best)
    return None, None, st, max_limit          # gave up at the depth cap
```

Visual Studio Code – real code – hospital_search.py (Lines 89–123)

IDDFS-tree is the pure recursive method analysed in Tasks 1–7. IDDFS-memo (memo=True) is the practical modification with the “deepest-remaining” table; it also stops by itself on an unreachable goal. The pure version stops at the depth cap or at a budget of 5,000,000 generated nodes.

8.3 Code: the experiment driver (run_experiments.py)



```python
import time, json, random, statistics as stt
from collections import deque
from hospital_search import *

SIZES = [1000, 10000, 100000, 1000000]
SEED = {1000: 11, 10000: 22, 100000: 33, 1000000: 44}     # graph seeds
GOAL_SEED = 2024                                           # seed for random start/goal picks
RUNS = 5
BUDGET = 5_000_000                                         # max nodes the pure IDDFS may generate

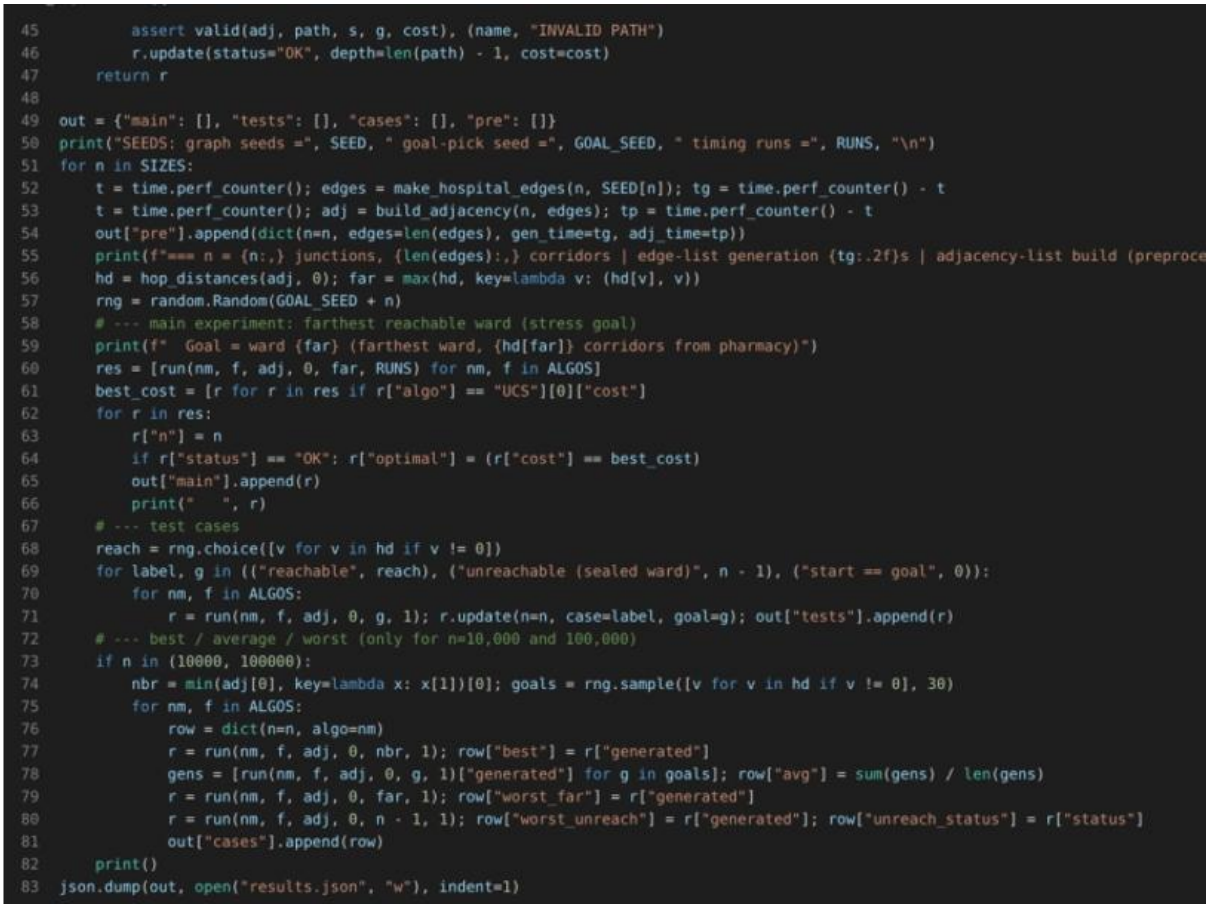
ALGOS = [("BFS",         lambda a, s, g: bfs(a, s, g)[:3]),
         ("UCS",         lambda a, s, g: ucs(a, s, g)[:3]),
         ("IDDFS-tree", lambda a, s, g: iddfs(a, s, g, max_limit=40, budget=BUDGET)[:3]),
         ("IDDFS-memo", lambda a, s, g: iddfs(a, s, g, max_limit=40, memo=True)[:3])]

def hop_distances(adj, s):
    d = {s: 0}; q = deque([s])
    while q:
        u = q.popleft()
        for v, _ in adj[u]:
            if v not in d: d[v] = d[u] + 1; q.append(v)
    return d

def valid(adj, path, s, g, cost):
    if path[0] != s or path[-1] != g: return False
    tot = 0
    for a, b in zip(path, path[1:]):
        w = [w for v, w in adj[a] if v == b]
        if not w: return False
        tot += w[0]
    return tot == cost

def run(name, fn, adj, s, g, reps):
    try:
        path, cost, st = fn(adj, s, g)
    except BudgetExceeded:
        return dict(algo=name, status="BUDGET", generated=BUDGET)
    times = []
    for _ in range(reps):
        t = time.perf_counter(); fn(adj, s, g); times.append(time.perf_counter() - t)
    r = dict(algo=name, generated=st.generated, expanded=st.expanded, max_frontier=st.max_frontier,
             time=sum(times) / len(times))
    if path is None: r.update(status="NO ROUTE")
    else:
```

Visual Studio Code – real code – run_experiments.py (Lines 1–44)



```
        assert valid(adj, path, s, g, cost), (name, "INVALID PATH")
        r.update(status="OK", depth=len(path) - 1, cost=cost)
    return r

out = {"main": [], "tests": [], "cases": [], "pre": []}
print("SEEDS: graph seeds =", SEED, " goal-pick seed =", GOAL_SEED, " timing runs =", RUNS, "\n")
for n in SIZES:
    t = time.perf_counter(); edges = make_hospital_edges(n, SEED[n]); tg = time.perf_counter() - t
    t = time.perf_counter(); adj = build_adjacency(n, edges); tp = time.perf_counter() - t
    out["pre"].append(dict(n=n, edges=len(edges), gen_time=tg, adj_time=tp))
    print(f"=== n = {n:,} junctions, {len(edges):,} corridors | edge-list generation {tg:.2f}s | adjacency-list build (preproce
    hd = hop_distances(adj, 0); far = max(hd, key=lambda v: (hd[v], v))
    rng = random.Random(GOAL_SEED + n)
    # --- main experiment: farthest reachable ward (stress goal)
    print(f"  Goal = ward {far} (farthest ward, {hd[far]} corridors from pharmacy)")
    res = [run(nm, f, adj, 0, far, RUNS) for nm, f in ALGOS]
    best_cost = [r for r in res if r["algo"] == "UCS"][0]["cost"]
    for r in res:
        r["n"] = n
        if r["status"] == "OK": r["optimal"] = (r["cost"] == best_cost)
        out["main"].append(r)
        print("   ", r)
    # --- test cases
    reach = rng.choice([v for v in hd if v != 0])
    for label, g in (("reachable", reach), ("unreachable (sealed ward)", n - 1), ("start == goal", 0)):
        for nm, f in ALGOS:
            r = run(nm, f, adj, 0, g, 1); r.update(n=n, case=label, goal=g); out["tests"].append(r)
    # --- best / average / worst (only for n=10,000 and 100,000)
    if n in (10000, 100000):
        nbr = min(adj[0], key=lambda x: x[1])[0]; goals = rng.sample([v for v in hd if v != 0], 30)
        for nm, f in ALGOS:
            row = dict(n=n, algo=nm)
            r = run(nm, f, adj, 0, nbr, 1); row["best"] = r["generated"]
            gens = [run(nm, f, adj, 0, g, 1)["generated"] for g in goals]; row["avg"] = sum(gens) / len(gens)
            r = run(nm, f, adj, 0, far, 1); row["worst_far"] = r["generated"]
            r = run(nm, f, adj, 0, n - 1, 1); row["worst_unreach"] = r["generated"]; row["unreach_status"] = r["status"]
            out["cases"].append(row)
    print()
json.dump(out, open("results.json", "w"), indent=1)
```

Visual Studio Code – real code – run_experiments.py (Lines 45–83)

8.4 Test inputs, expected behaviour and actual output

Test case	Expected behaviour
Reachable goal	All algorithms return a valid route. BFS and IDDFS give the route with the fewest corridors; UCS gives the cheapest route (cost ≤ the others).
Unreachable goal (sealed ward)	BFS and UCS finish after exploring the whole connected part and report “no route”. Pure IDDFS cannot prove it – it stops at the cap/budget (“gave up”). IDDFS-memo reports “no route”.
Start = goal	Route of length 0 and cost 0 with 1 node generated.

n	test case	algorithm	result	depth	cost	generated
1,000	reachable	BFS	route	3	11	149
1,000	reachable	UCS	route	3	11	267
1,000	reachable	IDDFS-tree	route	3	11	208
1,000	reachable	IDDFS-memo	route	3	11	199
1,000	unreachable (sealed ward)	BFS	no route	-	-	999
1,000	unreachable (sealed ward)	UCS	no route	-	-	1,119
1,000	unreachable (sealed ward)	IDDFS-tree	gave up	-	-	> 5,000,000
1,000	unreachable (sealed ward)	IDDFS-memo	no route	-	-	11,552
1,000	start == goal	BFS	route	0	0	1
1,000	start == goal	UCS	route	0	0	1
1,000	start == goal	IDDFS-tree	route	0	0	1
1,000	start == goal	IDDFS-memo	route	0	0	1
10,000	reachable	BFS	route	7	31	5,985
10,000	reachable	UCS	route	7	24	7,470
10,000	reachable	IDDFS-tree	route	7	31	14,743
10,000	reachable	IDDFS-memo	route	7	31	11,479
10,000	unreachable (sealed ward)	BFS	no route	-	-	9,999
10,000	unreachable (sealed ward)	UCS	no route	-	-	11,187
10,000	unreachable (sealed ward)	IDDFS-tree	gave up	-	-	> 5,000,000
10,000	unreachable (sealed ward)	IDDFS-memo	no route	-	-	180,039
10,000	start == goal	BFS	route	0	0	1
10,000	start == goal	UCS	route	0	0	1
10,000	start == goal	IDDFS-tree	route	0	0	1
10,000	start == goal	IDDFS-memo	route	0	0	1
100,000	reachable	BFS	route	7	38	17,227
100,000	reachable	UCS	route	7	25	19,835
100,000	reachable	IDDFS-tree	route	7	38	27,799
100,000	reachable	IDDFS-memo	route	7	38	26,471
100,000	unreachable (sealed ward)	BFS	no route	-	-	99,999
100,000	unreachable (sealed ward)	UCS	no route	-	-	111,984

Visual Studio Code terminal – real output – output of report_tests.py (Part 1)

```
$ python3 report_tests.py (continued)
  100,000  unreachable (sealed ward) IDDFS-tree gave up       -     - > 5,000,000
  100,000  unreachable (sealed ward) IDDFS-memo no route      -     -   1,975,284
  100,000  start == goal             BFS        route        0     0           1
  100,000  start == goal             UCS        route        0     0           1
  100,000  start == goal             IDDFS-tree route        0     0           1
  100,000  start == goal             IDDFS-memo route        0     0           1
1,000,000  reachable                 BFS        route        5    26       4,335
1,000,000  reachable                 UCS        route        5    26     118,297
1,000,000  reachable                 IDDFS-tree route        5    26       5,439
1,000,000  reachable                 IDDFS-memo route        5    26       5,435
1,000,000  unreachable (sealed ward) BFS        no route     -     -     999,999
1,000,000  unreachable (sealed ward) UCS        no route     -     -   1,121,089
1,000,000  unreachable (sealed ward) IDDFS-tree gave up      -     - > 5,000,000
1,000,000  unreachable (sealed ward) IDDFS-memo no route     -     -  24,167,802
1,000,000  start == goal             BFS        route        0     0           1
1,000,000  start == goal             UCS        route        0     0           1
1,000,000  start == goal             IDDFS-tree route        0     0           1
1,000,000  start == goal             IDDFS-memo route        0     0           1
```

Visual Studio Code terminal – real output – output of report_tests.py (Part 2)

Result. All three test types behave as expected at all four sizes. For reachable goals all algorithms return the same depth; UCS's cost is never larger (for example n = 10,000: BFS cost 31, UCS cost 24). For the sealed ward only the pure IDDFS fails to finish – exactly as the theory in Task 4 says.

Evidence for Task 8: working code, test inputs, expected behaviour, actual output, seeds, and a reproducible procedure. The code is the same as the algorithms analysed in Tasks 1–7.

Task 9 – Record Experimental Performance

For the stress goal (the ward farthest from the pharmacy) every algorithm was run on the same graph. The count of nodes, frontier, depth and cost come from one run; time is the average of 5 runs (time.perf_counter). For IDDFS, "max frontier" is the deepest recursion (for IDDFS-memo it also includes the size of its table). "Optimal" means the path cost equals the UCS cost.

TABLE A - farthest ward from the pharmacy (time = mean of 5 runs)

n	algorithm	generated	expanded	max frontier	depth	cost	optimal	time (s)
1,000	BFS	998	983	344	8	39	NO	0.0003
1,000	UCS	1,115	974	391	9	30	yes	0.0007
1,000	IDDFS-tree	35,172	12,702	8	8	39	NO	0.0102
1,000	IDDFS-memo	7,594	4,826	1,001	8	39	NO	0.0038
10,000	BFS	9,999	9,989	3,268	11	49	yes	0.0056
10,000	UCS	11,186	9,990	3,713	11	49	yes	0.0133
10,000	IDDFS-tree	1,701,373	581,846	11	11	49	yes	0.7704
10,000	IDDFS-memo	129,079	91,414	10,006	11	49	yes	0.1022
100,000	BFS	99,982	99,843	31,432	13	68	NO	0.1719
100,000	UCS	111,895	99,242	36,659	13	50	yes	0.3819
100,000	IDDFS-tree	> 5,000,000	-	-	-	-	-	(stopped)
100,000	IDDFS-memo	1,320,893	918,810	99,990	13	68	NO	2.1971
1,000,000	BFS	999,991	999,877	313,618	15	82	NO	3.1700
1,000,000	UCS	1,121,087	999,976	368,047	18	69	yes	8.0682
1,000,000	IDDFS-tree	> 5,000,000	-	-	-	-	-	(stopped)
1,000,000	IDDFS-memo	17,873,963	13,293,073	999,993	15	82	NO	39.7538

PREPROCESSING (done once)

n	corridors	edge list (s)	adjacency list (s)
1,000	1,496	0.002	0.000
10,000	14,995	0.038	0.014
100,000	149,997	0.151	0.073
1,000,000	1,499,995	3.658	1.989

Visual Studio Code terminal – real output – output of report_tables.py

Notes. (1) "> 5,000,000 (stopped)" means the pure IDDFS reached the budget and was stopped, so there is no final answer; that is itself a result. (2) BFS returned a non-optimal route for 3 of the 4 sizes – it finds the fewest corridors, not the lowest time. (3) Preprocessing (building the adjacency list) is shown separately and must be added to every algorithm's time.

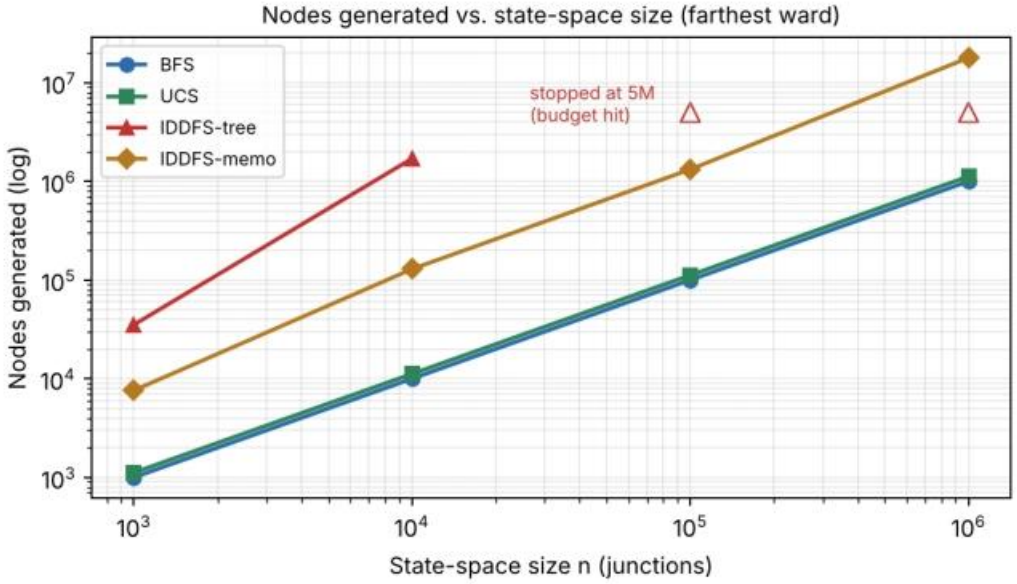


Figure 3 – Nodes generated against state-space size (both axes log scale).

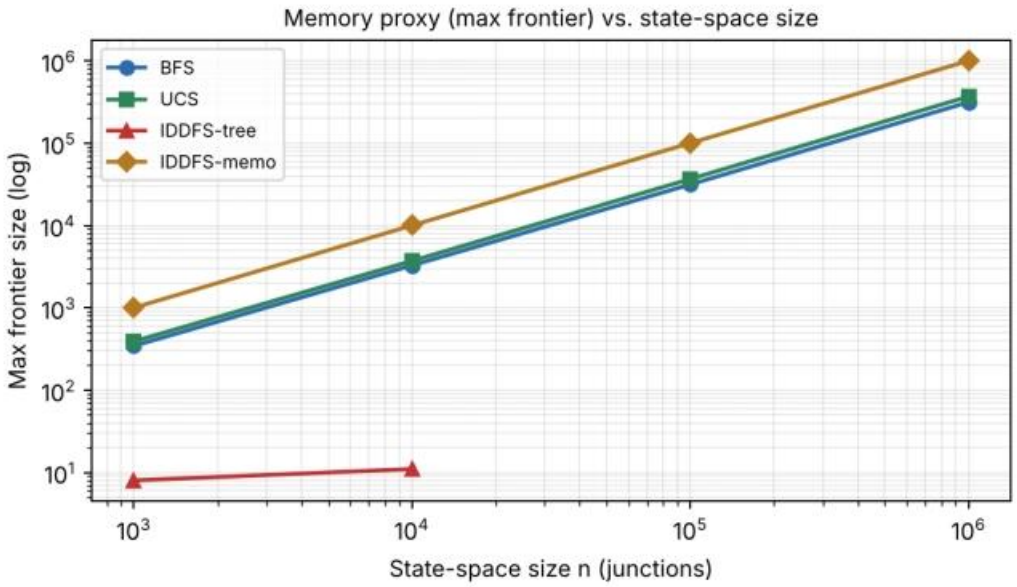


Figure 4 – Maximum frontier size (memory proxy) against state-space size (log scale).

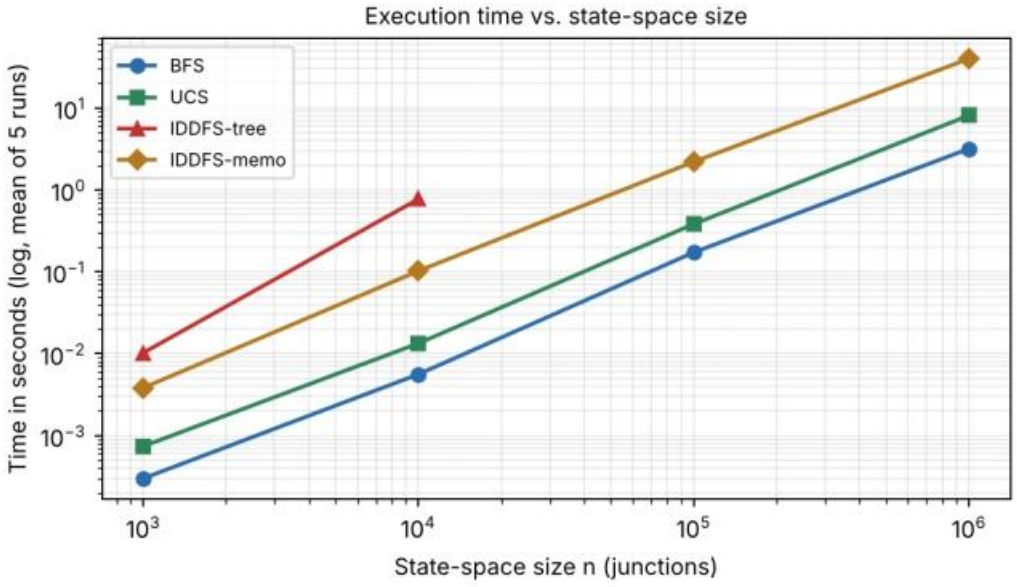


Figure 5 – Average execution time of 5 runs against state-space size (log scale).

Evidence for Task 9: table and three log-scale graphs for nodes generated and expanded, max frontier, depth, cost, optimality and average time (5 runs), with the code and real output above.

Task 10 – Compare Theory with Experiment

10.1 Does the growth agree with the theory?

Algorithm	Theory says	Experiment shows	Agree?
BFS nodes	Θ(V+E): grows ×10 when V grows ×10	998 → 9,999 → 99,982 → 999,991 (almost exactly ×10 each time; equal to V)	Yes

BFS frontier	Θ(V)	344 → 3,268 → 31,432 → 313,618: always about 0.31–0.34 V	Yes
UCS nodes	O((V+E) log V); a little more pushes than V	1,115 → 11,186 → 111,895 → 1,121,087: constant 1.12 V; time 2.2–2.5× BFS (the log factor and heap)	Yes
IDDFS-pure nodes	$\Theta(b^d)$: exponential in depth	35,172 → 1,701,373 (×48 for 3 more levels, so b ≈ 3.6) → over 5,000,000. Memory only 8 → 11	Yes
IDDFS-pure memory	Θ(d)	Max recursion depth 8, 11 (equals the solution depth)	Yes
IDDFS-memo nodes	O(d (V+E))	7.6 V, 12.9 V, 13.2 V, 17.9 V while d = 8, 11, 13, 15: grows with d	Yes

10.2 Which algorithm scales best?

BFS scales best in raw work (fewest nodes, 3.2 s at 1,000,000 states). UCS is next (8.1 s, 2.5× slower) but it is the only one that always returns the cheapest route. IDDFS-memo needed 39.8 s at 1,000,000 states (12× slower than BFS) because every iteration repeats the whole search. IDDFS-pure does not scale at all: it passes 5,000,000 nodes already at 100,000 junctions.

10.3 Differences between theory and observation, and the reasons

- **Finite graph and explored set.** The tree formula b^d (and $b^{1+\lfloor C^*/\varepsilon \rfloor}$ for UCS) is huge – for C* = 69, ε = 1 it is astronomical – but real BFS/UCS never exceed about V nodes because duplicates are removed. So the tree formulas are only loose upper limits for these two.
- **Graph structure.** The generator creates a few hub junctions with many corridors, so the average branching b is about 3 but the effective branching along paths is higher (we measured b ≈ 3.6 for IDDFS). The small diameter (8–15 corridors even at 1,000,000 junctions) keeps d small, which is why BFS finds the goal quickly in the average case.
- **IDDFS-memo is above V.** Theory for the memo table suggests about d·V, but the table allows re-expansion when a junction is reached again with more remaining depth, so the count goes beyond V (up to 17.9 V).
- **Tie-breaking.** The order of corridors in the adjacency list decides which neighbour is explored first. This changes the best-case and the IDDFS counts but not the order of growth.
- **Language overhead.** Python dictionaries, sets and the heap have large constants; the heap adds the log factor for UCS. Absolute times would be many times smaller in C++, but the **growth shapes** would stay the same.
- **BFS is not optimal.** The theory says BFS finds the fewest-corridor route; the data confirm that this route can be much more expensive in time (cost 82 vs 69 at 1,000,000 states, 68 vs 50 at 100,000).

Evidence for Task 10: comparison of observed growth with each theoretical function, scaling ranking, and reasoned explanation of differences (graph structure, duplicates, tie-breaking, language overhead).

Task 11 – Recommend and Defend the Final Design

Recommendation: Uniform-Cost Search with an explored set (graph search), using the pre-built adjacency list. Use BFS only as a quick helper when all corridors cost the same.

11.1 Why UCS and not the others

- **It answers the real question.** The robot should deliver as fast as possible, and its performance measure is travel time, not the number of corridors. UCS is the only one of the three that is **guaranteed optimal** in time. BFS gave a route that was longer in time in 3 of 4 tests (up to 36% longer: 68 versus 50 at 100,000 states). IDDFS has the same weakness as BFS.
- **Theory + experiment agree.** Theory: O((V+E) log V) time and Θ(V) memory. Experiment: 0.38 s at 100,000 states and 8.1 s at 1,000,000 states in plain Python, about 1.12 V nodes, frontier 368,047 – and the same cost as the best route in every test.
- **Not just the lowest Big-O.** BFS has a slightly lower Big-O and is 2.5× faster, but its answer is wrong for our goal. IDDFS has the lowest memory, but its time is exponential (over 5,000,000 nodes at 100,000 states) and it cannot say "no route". The memo version fixes that but is 12× slower than BFS and gives up the memory advantage.

11.2 Preprocessing, trade-offs and limitations

- **Preprocessing:** the adjacency list takes 0.07 s (100,000 states) to 1.99 s (1,000,000 states). It is built **once** when the map is loaded, not for every delivery.
- **Because every trip starts at the pharmacy**, UCS can be run once from the pharmacy and its parent pointers saved; every ward request is then answered by simply following the saved pointers (cost proportional to the path length). The search is re-run only when the map changes (new wing, closed corridor, new travel times).
- **Memory:** Θ(V). At 1,000,000 states the frontier holds 368,047 entries and the explored set about 1,000,000, which is fine for a server but heavy for a small on-board computer. If memory is very tight, the route table can be computed on a server and only the route is sent to the robot.
- **Limitations:** it assumes the map and travel times are known and fixed; moving obstacles need re-planning; and it needs to know where the robot is (see Day 4).

11.3 Is it still practical as the hospital grows?

Yes. Growth is almost linear: ×10 states gives about ×10 nodes and about ×20 time (from the log factor and cache effects). A million-junction hospital is solved in about 8 s in Python (and well below a second in a compiled language), one time per map version. Exponential methods, by contrast, are already unusable at 100,000 states.

Evidence for Task 11: recommendation based on our own tables – theory (Tasks 3, 5) and experiment (Task 9) – with preprocessing, trade-offs, limitations and practicality for growth.

Task 12 – Day-4 Design Challenge: Re-evaluate and Adapt the Design

Changed requirement: after a power restart or sensor fault the robot knows only that it is at one of k possible junctions. It must still reach the pharmacy docking station.

A. Impact analysis

What has changed

- Before: the robot knew its start junction exactly (known initial state) and could see where it was after each move (fully observable).
- Now: the start is only known to be one of k junctions, and the position is not reliably observed. The problem becomes partially observable (without a sensor it is a "sensorless/conformant" problem).
- Two original assumptions are broken: (1) "the start state is known", (2) "the current state is always observed". The graph, the corridor costs and the goal (the pharmacy) are unchanged.

Why the robot's knowledge becomes part of the state. The right action depends on what the robot believes, not only on where it really is. For example, "turn left" may be correct if it is at junction 5 but wrong at junction 9. A single plan must work for all junctions that are still possible. Also, actions change the knowledge: moving into a wall (a "bump") can remove some possibilities. So the planner has to track **the set of junctions that are still possible**. This set is called a **belief state**, and the search must be done over belief states.

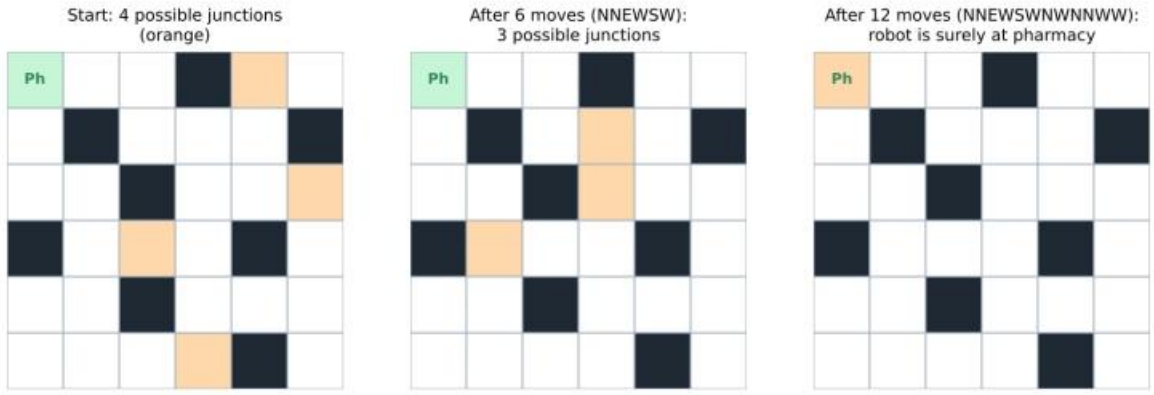


Figure 6 – Belief states on a small ward-block floor plan (orange = possible positions, dark = wall). The real plan found by the code shrinks 4 possibilities to 1: NNEWSWNWNNWW.

B. Evaluation of the original design

- **Can the original algorithm be used?** Not directly. BFS/UCS need a single known start state. Running them from each of the k possible starts gives k different routes, and the robot does not know which one to follow.
- **What a state becomes.** A state is now a belief state: a subset of junctions (the possible locations). Initial belief = the k possible junctions. Action = a move command. Transition: apply the move to every junction in the set (RESULT(B, a) = {result(s, a) : s ∈ B}). Goal test: belief = {pharmacy}, i.e. the robot is sure it is at the pharmacy.

- **How big is the new space?** In general a belief is any non-empty subset, so up to $2^V - 1$ beliefs. Because actions are deterministic, a set never becomes bigger, so beliefs reachable from a start of size k have at most $\Sigma_{i=1}^{k} C(V, i) = O(V^k/k!)$ beliefs.

V (junctions)	**k = 2**	**k = 4**	**k = 8**
28 (our test floor plan)	406	24,157	4,791,322
1,000	$\approx 5.0 \times 10^5$	$\approx 4.2 \times 10^{10}$	$\approx 2.5 \times 10^{19}$
1,000,000	$\approx 5.0 \times 10^{11}$	$\approx 4.2 \times 10^{22}$	$\approx 2.5 \times 10^{43}$

Is the original design acceptable? No. The original had V states (1,000,000 at most); the new search space is up to about $V^k/k!$. Even for k = 2 and a hospital of 1,000,000 junctions this is about 5×10^{11} belief states, so searching blindly over beliefs with BFS/UCS is not practical at hospital scale. We need to **modify** the design.

C. Adaptation or redesign

Decision: MODIFY. The hospital graph, the cost model, and the BFS/UCS machinery (frontier, explored set) are kept. What changes is the definition of a state and the use of sensing.

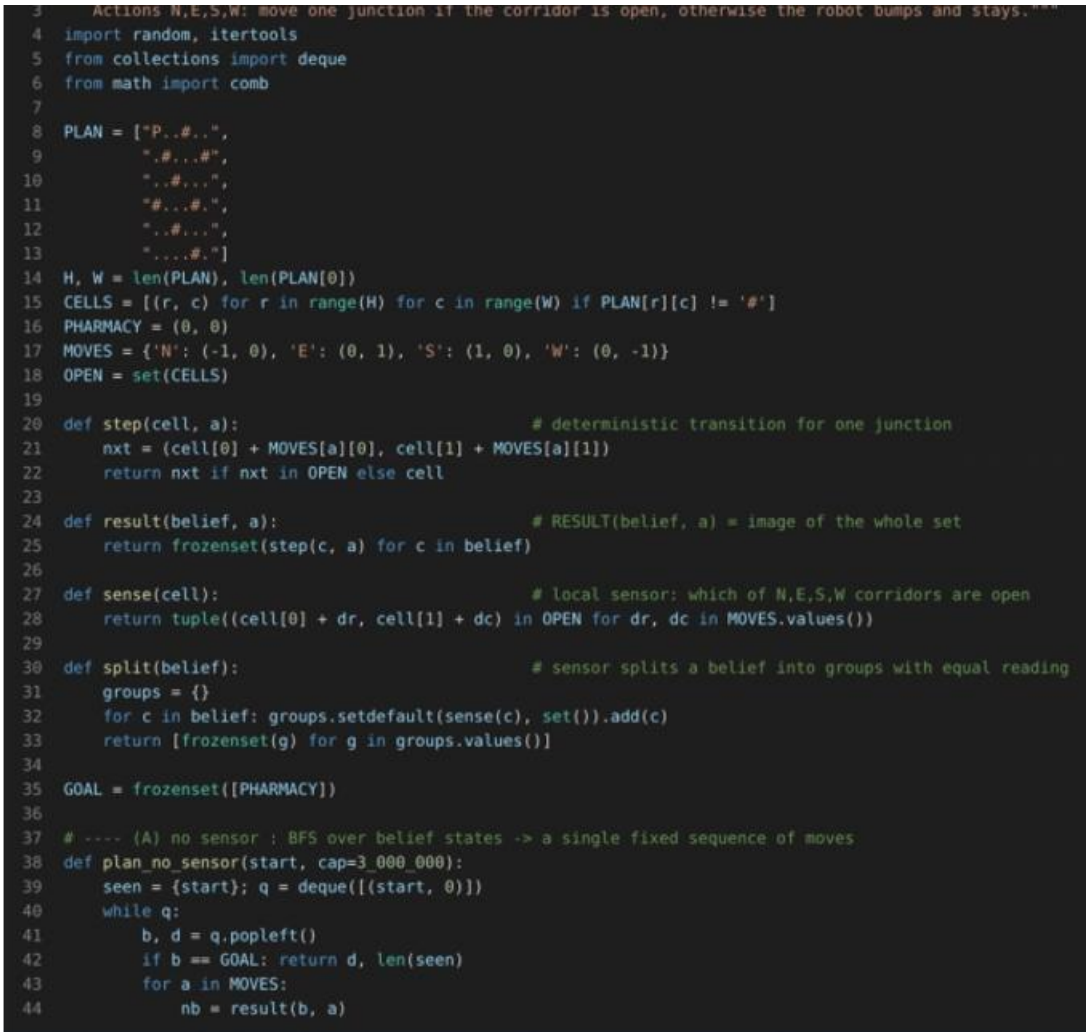
1. **Search over belief states (no sensor).** Run BFS over beliefs with the actions described above. The answer is one fixed list of moves that brings every possible start to the pharmacy. Worst-case size $O(V^k/k!)$ – exact for small blocks, but too big for the whole hospital.
2. **Use a local sensor and a conditional plan (AND-OR search).** After each move the sensor gives a reading (here: which of the four corridors N, E, S, W are open at the current junction). The reading **splits** the belief into smaller beliefs (one per reading). The plan becomes a tree: for each reading the robot has a different next move. The search is an AND-OR search – OR over the move to choose, AND over all possible readings. Because each reading cuts the belief, the number of beliefs stays small.
3. **In the real hospital (V = 1,000,000):** use the sensor/landmarks (junction type, ward tag, lift door) to shrink the belief to one junction first; as soon as the belief has size 1 the robot **switches back to the original UCS** from that junction to the pharmacy. So the original solution is reused after localisation, and the expensive belief-space search only runs on a small area.

Effect on complexity. Without a sensor: belief states up to $\Sigma\, C(V, i) \le O(V^k/k!)$ – polynomial of degree k in V. With a sensor: each move can split a belief into at most $2^4 = 16$ readings, but the sets get smaller and smaller, so the number of beliefs reached is close to the number of single junctions (measured next). After localisation: the same $O((V+E) \log V)$ as before.

D. Evidence-based defence

Experiment. A ward block of 28 open junctions (6 × 6 floor plan, walls shown as #; Ph = pharmacy docking station at the top-left). Actions are N, E, S, W; a move into a wall is a "bump" (the robot stays). For each k = 1, 2, 4, 8 we pick 10 random sets of k possible junctions (seeds $100 \cdot s + k$, s = 1…10) and report the average number of belief states generated and the plan length. Plan length = number

of moves in the plan (for the sensor version: the worst case over all sensor readings). Here k = 1 is the original problem.



```
    Actions N,E,S,W: move one junction if the corridor is open, otherwise the robot bumps and stays."""
import random, itertools
from collections import deque
from math import comb

PLAN = ["P..#..",
        ".#...#",
        "..#...",
        "#...#.",
        "..#...",
        "....#."]
H, W = len(PLAN), len(PLAN[0])
CELLS = [(r, c) for r in range(H) for c in range(W) if PLAN[r][c] != '#']
PHARMACY = (0, 0)
MOVES = {'N': (-1, 0), 'E': (0, 1), 'S': (1, 0), 'W': (0, -1)}
OPEN = set(CELLS)

def step(cell, a):                          # deterministic transition for one junction
    nxt = (cell[0] + MOVES[a][0], cell[1] + MOVES[a][1])
    return nxt if nxt in OPEN else cell

def result(belief, a):                      # RESULT(belief, a) = image of the whole set
    return frozenset(step(c, a) for c in belief)

def sense(cell):                            # local sensor: which of N,E,S,W corridors are open
    return tuple((cell[0] + dr, cell[1] + dc) in OPEN for dr, dc in MOVES.values())

def split(belief):                          # sensor splits a belief into groups with equal reading
    groups = {}
    for c in belief: groups.setdefault(sense(c), set()).add(c)
    return [frozenset(g) for g in groups.values()]

GOAL = frozenset([PHARMACY])

# ---- (A) no sensor : BFS over belief states -> a single fixed sequence of moves
def plan_no_sensor(start, cap=3_000_000):
    seen = {start}; q = deque([(start, 0)])
    while q:
        b, d = q.popleft()
        if b == GOAL: return d, len(seen)
        for a in MOVES:
            nb = result(b, a)
```

Visual Studio Code – real code – belief_search.py (Lines 1–44)

```
45              if nb not in seen:
46                  seen.add(nb); q.append((nb, d + 1))
47                  if len(seen) > cap: return None, len(seen)
48      return None, len(seen)                      # no sequence works
49
50  # ---- (B) local sensor : AND-OR search, iterative deepening on the worst-case plan length
51  def plan_with_sensor(start):
52      seen, memo = set(), {}
53      def solvable(b, depth):                     # can the robot dock within `depth` moves, whatever it senses?
54          if b == GOAL: return True
55          if depth == 0: return False
56          key = (b, depth)
57          if key in memo: return memo[key]
58          memo[key] = False                       # (loop guard)
59          ok = False
60          for a in MOVES:                         # OR node: choose an action
61              nb = result(b, a)
62              parts = split(nb)                   # AND node: every sensor reading must be solved
63              seen.update(parts)
64              if all(solvable(p, depth - 1) for p in parts): ok = True; break
65          memo[key] = ok
66          return ok
67      groups = split(start); seen.update(groups)
68      for D in range(0, 60):
69          if all(solvable(g, D) for g in groups): return D, len(seen)
70      return None, len(seen)
71
72  if __name__ == "__main__":
73      print("Floor plan:"); [print("  ", row) for row in PLAN]
74      n = len(CELLS); print(f"Open junctions n = {n}\n")
75      SEEDS = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
76      print(f"{'k':>2} | {'no-sensor: plan len':>19} {'belief states':>14} | {'sensor: worst plan len':>22} {'belief states':>14
77      for k in (1, 2, 4, 8):
78          A, B = [], []
79          for sd in SEEDS:
80              start = frozenset(random.Random(sd * 100 + k).sample(CELLS, k))
81              A.append(plan_no_sensor(start)); B.append(plan_with_sensor(start))
82          avg = lambda xs: sum(xs) / len(xs)
83          bound = sum(comb(n, i) for i in range(1, k + 1))
84          print(f"{k:>2} | {avg([a[0] for a in A]):>19.1f} {avg([a[1] for a in A]):>14.0f} | "
85                f"{avg([b[0] for b in B]):>22.1f} {avg([b[1] for b in B]):>14.0f} | {bound:,}")
```

Visual Studio Code – real code – belief_search.py (Lines 45–85)

```
Floor plan:
   P..#..
   .#...#
   ..#...
   #...#.
   ..#...
   ....#.
Open junctions n = 28

 k | no-sensor: plan len  belief states | sensor: worst plan len  belief states | bound sum C(n,i)
 1 |                5.6             22 |                    5.6             19 | 28
 2 |                8.4            305 |                    7.4             27 | 406
 4 |                9.8           8240 |                    8.2             29 | 24,157
 8 |               11.6          96443 |                    8.9             31 | 4,791,322
```

Visual Studio Code terminal – real output – output of belief_search.py

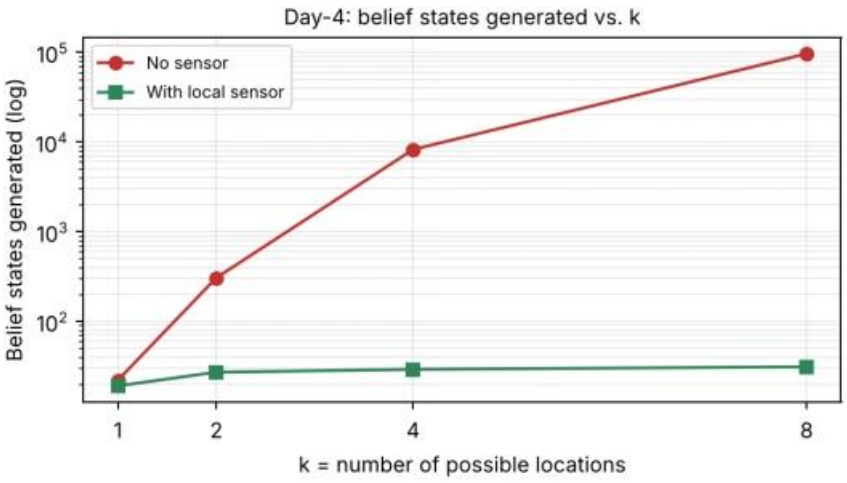


Figure 7 – Belief states generated for k = 1, 2, 4, 8 with and without a local sensor (log scale).

What the evidence shows

- **Without a sensor** the number of belief states grows very fast: 22 → 305 → 8,240 → 96,443 for k = 1, 2, 4, 8 (about ×14, ×27 and ×12 as k doubles), following the polynomial bound Σ C(28, i) (28 → 406 → 24,157 → 4,791,322). The plan also becomes longer (5.6 → 11.6 moves).
- **With the local sensor** the beliefs stay almost flat: 19 → 27 → 29 → 31, and the plan is shorter (5.6 → 8.9 moves). For k = 8 the sensor needs about 3,000 times fewer belief states (31 versus 96,443) and 2.7 fewer moves.
- For k = 1 both give the original shortest route (5.6 moves on average), so the new method contains the old one as a special case.
- **Decision confirmed:** retain the graph and algorithm ideas, but modify the state to a belief state and use the sensor to keep it small; once the robot is localised, return to the original UCS.

Limitation of the experiment: only a small 28-junction block with deterministic moves and one type of sensor, averaged over 10 random start sets; the numbers will differ on other maps, but the trend (exponential-like growth without a sensor, nearly flat with a sensor) is what the mathematics predicts.

Evidence for Task 12: A impact analysis, B evaluation with the size of the new search space, C modified strategy and its complexity, D code, real output, table and graph for k = 2, 4, 8 with and without a local sensor.

Final Summary

- Three different uninformed searches were designed: BFS, UCS (iterative) and IDDFS (recursive).
- Maths: BFS $\Theta(V+E)$; UCS $O((V+E) \log V)$; IDDFS $\Theta(b^d)$ time but only $\Theta(d)$ memory. Recurrence $T(l) = b \cdot T(l-1) + c$, $T(0) = 1$ solves to $\Theta(b^l)$.
- Experiments up to 1,000,000 states agree with the theory. Recommendation: UCS, because it gives the cheapest travel time and scales almost linearly.
- Day 4: when the position is uncertain, plan over belief states and use the local sensor; after localisation return to UCS.